

Comprehensive Documentation of the Skylens Project

From Raw Data Storage to Interactive Dashboards — Architecture, Tools, Implementation Steps,
and the Problems We Faced (with Solutions)

Contents

1	General Introduction	2
1.1	The Data Used	2
2	Project Objectives	3
3	Tools and Technologies Used (Tech Stack)	4
3.1	Why This Particular Combination of Tools?	4
4	Overall Pipeline Architecture	6
5	Detailed Walkthrough of the Airflow DAG Steps	7
5.1	Task 1: check_raw_data	7
5.2	Task 2: spark_row_count	7
5.3	Task 3: clean_flight_data	7
5.4	Task 4: add_derived_columns	8
5.5	Task 5: generate_aggregated_tables	8
5.6	Task 6: load_to_postgres	8
6	The Resulting Analytical Tables in PostgreSQL	10
7	Connecting the Database to BI Tools	11
7.1	Connecting to Power BI	11
7.2	Connecting to Apache Superset	11
8	The Dashboard and the Five Charts	13
8.1	Notes on Some of the Charts	13
9	Main Problems We Faced and How We Solved Them	14
9.1	Problem 1: load_to_postgres Task Failing With ClassNotFoundException	14
9.2	Problem 2: Could not load database driver for: postgresql Error in Superset	15
9.3	Problem 3: SSL Error When Connecting Superset to PostgreSQL	16
9.4	Problem 4: Errors Building the First Charts (Mixing Up X-axis, Metrics, and Dimensions)	16
9.5	Problem 5: Duplicate column/metric labels Error	17
10	Summary of General Lessons Learned	18

1 General Introduction

Skylens is a full end-to-end Big Data project that builds a complete data pipeline for analyzing airline flight data — starting from ingesting raw data, all the way to interactive dashboards that any non-technical person can open and use to make decisions.

The core idea of the project is to simulate a realistic Data Engineer workflow, where the job is not just writing code that processes data, but also:

- Storing raw data in a distributed storage system.
- Cleaning and processing it at scale using a parallel processing engine.
- Scheduling and orchestrating every step automatically, with no manual intervention.
- Producing ready-made aggregated analytical tables.
- Loading the results into a relational database that acts as a Data Warehouse.
- Connecting that warehouse to Business Intelligence tools such as Power BI and Apache Superset to visualize the results.

All components of the project run inside Docker containers orchestrated by a single `docker-compose.yml` file, forming one integrated environment that can be spun up with a single command on any machine that has Docker installed.

1.1 The Data Used

The core dataset is historical domestic airline on-time performance data (similar to the well-known “Airline On-Time Performance” datasets), and includes information such as:

- Flight date, reporting airline, flight number.
- Origin and destination airports and their associated cities.
- Scheduled and actual departure/arrival times.
- Departure and arrival delay durations, and their different causes (carrier delay, weather, National Airspace System, security, late aircraft).
- Flight status (cancelled or diverted or not).
- Flight distance in miles.

The raw data volume is fairly large (CSV files totaling roughly 8.7 GB, with more than 20 million rows), which is exactly why distributed processing tools were required instead of relying on traditional single-machine tools like Pandas alone.

2 Project Objectives

The objectives of the Skylens project can be summarized as follows:

1. **Build a fully integrated Big Data environment locally:** run every component (storage, processing, scheduling, databases, visualization) inside Docker containers orchestrated together.
2. **Fully automate the processing pipeline:** every step executes automatically, in the correct order, with no manual intervention, using a scheduling tool (Apache Airflow).
3. **Process data at scale:** clean the data, detect invalid rows, remove duplicates, and derive additional columns that help later analysis — all using Apache Spark instead of tools that load the entire dataset into memory.
4. **Produce ready-to-use analytical tables:** instead of leaving analysts to work with 20 million raw rows, the project computes five aggregated tables ready for direct use in reports.
5. **Load the results into a Data Warehouse:** using PostgreSQL as the final database that any visualization tool can easily connect to.
6. **Connect to Business Intelligence tools:** linking this warehouse to both Power BI (Microsoft's desktop tool) and Apache Superset (an open-source web-based tool) to visualize the results interactively.

3 Tools and Technologies Used (Tech Stack)

The project relies on a set of well-known open-source Big Data tools, all orchestrated through Docker Compose on a single internal network called `bigdata-net`. The table below summarizes each tool and its role:

Tool / Service	Role in the Project
Docker & Docker Compose	Runs and orchestrates all services as separate containers on one internal network, and manages persistent storage (Volumes) for each service
HDFS (namenode + datanode, images <code>bde2020/hadoop-namenode</code> and <code>bde2020/hadoop-datanode</code>)	Distributed file system used to store the raw flight data (CSV) and the processed data (Parquet)
Apache Spark (Standalone cluster: one <code>spark-master</code> + one <code>spark-worker</code>)	The parallel processing engine: reads the raw data, cleans it, transforms it, and computes the analytical tables
Apache Airflow (<code>airflow-webserver</code> + <code>airflow-scheduler</code> , with a separate metadata database <code>airflow-postgres</code>)	Schedules and orchestrates the entire pipeline via a single DAG made up of 6 sequential tasks
PostgreSQL — <code>skylens-dw</code> (image <code>postgres:13</code>)	The final Data Warehouse holding the five ready-to-visualize analytical tables
Apache Kafka (KRaft mode, no Zookeeper) + Kafka UI	Part of the project's general infrastructure (streaming data), available in the environment for future extension
Jupyter Notebook	An interactive environment for exploring data and testing PySpark code independently of the DAG
Microsoft Power BI	An external (desktop) visualization tool that connects directly to the <code>skylens_dw</code> database over a port exposed to the host machine
Apache Superset	An open-source web-based visualization tool running inside the same internal network, added later to the project to build an interactive dashboard

3.1 Why This Particular Combination of Tools?

- **HDFS** is used as the initial storage layer because it is the natural default choice alongside any traditional Hadoop/Spark environment, and it theoretically allows files to be distributed across more than one storage node — even though this environment uses a single `datanode` for learning/experimentation purposes.
- **Apache Spark** was chosen as the processing engine because it works with data as a distributed `DataFrame`, backed by the Catalyst query optimizer, which reduces the need for custom Python functions (UDFs) or calls like `.collect()` that would pull the entire dataset into memory — something critical when dealing with 20 million rows.
- **Apache Airflow** was chosen as the orchestrator because it's the industry-standard tool for scheduling data pipelines, letting task dependencies be defined clearly, monitoring every run, and automatically retrying on failure.
- **PostgreSQL** was chosen as the final data warehouse because it's a mature relational database, supported by virtually every BI tool (including Power BI and Superset), and easy to run inside a Docker container.
- **Power BI and Superset together**: two different visualization tools were deliberately used to demonstrate how the same data warehouse (`skylens_dw`) can be consumed simultaneously by more than one BI tool

— one commercial, desktop-based (Power BI), and one open-source, web-based, reachable by anyone on the team from a browser (Superset).

4 Overall Pipeline Architecture

The data flow of the project can be summarized in the following ordered stages:

1. **Raw CSV files** → stored inside HDFS (path raw).
2. **Spark**: cleans the data and removes duplicates → separate Parquet files (cleaned / rejected).
3. **Spark**: adds derived columns (enriched), partitioned by Year and Month.
4. **Spark**: computes five aggregated analytical tables (aggregations).
5. **Spark (JDBC)**: loads the five tables into **PostgreSQL** (skylens_dw).
6. From skylens_dw, the data branches out to two visualization tools at once:
 - **Power BI** — direct connection over port 5433.
 - **Apache Superset** — internal connection over the Docker internal network.

This entire flow is managed by a **single Airflow DAG** named `skylens_batch_pipeline`, made up of 6 sequential tasks, so that no task starts until the one immediately before it has succeeded:

```
check_raw_data >> spark_row_count >> clean_flight_data
    >> add_derived_columns >> generate_aggregated_tables
    >> load_to_postgres
```

This linear DAG ordering makes sense here because each stage genuinely depends on the output of the previous one; for example, the analytical tables cannot be computed before the derived columns are added, and the derived columns cannot be added before the raw data is cleaned first.

5 Detailed Walkthrough of the Airflow DAG Steps

This section explains every task in the DAG in detail: its purpose, its internal logic, and why it was written the way it was.

5.1 Task 1: check_raw_data

Purpose: Confirm that the raw data files actually exist inside HDFS before continuing with the rest of the pipeline, to avoid wasting time and processing on an empty or missing dataset.

How it works: This task connects to HDFS via the `hdfs` library (not through Spark), using an `InsecureClient` to reach the namenode on port 9870, then lists the contents of the path `/skylens/raw/flights`. If the listing is empty, the task deliberately fails with a clear error (`ValueError`), which prevents Airflow from continuing to the downstream tasks.

Why not use Spark here? Because the check is very simple (just confirming files exist), and using the lightweight HDFS library is far faster than spinning up a full Spark session for such a small task.

5.2 Task 2: spark_row_count

Purpose: Confirm that Spark itself can connect to HDFS and correctly read/parse the files, before moving on to the heavier processing stages.

How it works: This task creates a Spark session (`SparkSession`) connected to the Spark cluster via `spark://spark-master:7077`, and reads CSV files from `hdfs://namenode:9000/skylens/raw/flights`. Instead of counting every row (which could take a long time on 20+ million rows), it only reads a **single row** (`df.limit(1).collect()`) to confirm the read works without errors. It then writes a simple text summary (`summary.csv`) to the processed path inside HDFS, directly through the HDFS client rather than through Spark, to avoid a known compatibility issue between `cloudpickle` and Python 3.11 when using `spark.createDataFrame` in this specific environment.

Key lesson here: this task is a good example of a “lightweight sanity check” before running expensive operations, rather than discovering a connectivity problem after an hour of real processing.

5.3 Task 3: clean_flight_data

This is the first genuinely heavy processing task in the pipeline, and it performs several sub-steps:

1. **Defining an explicit schema for the 29 core columns** instead of relying on `inferSchema=true`, which is very slow on an 8.7 GB file since it requires reading the file twice (once to infer types, once to actually read it).
2. **Selecting only the columns that actually exist:** some yearly files may contain slightly different extra columns, so the intersection of the required columns and the columns actually present in each file is selected.
3. **Casting numeric columns:** all numeric columns arrive as strings from CSV, so they’re explicitly cast to `DoubleType`.
4. **Validity check:** data is split into valid and rejected rows based on a logical condition — both the origin and destination airports and the flight date must be non-null, and distance (if present) must be greater than zero.
5. **Deduplication:** duplicate rows are removed based on a composite logical key combining flight date, reporting airline, flight number, origin, destination, and scheduled departure time.

6. **Writing as Parquet files:** valid rows are saved to `processed/flights/cleaned` and rejected rows to `rejected/flights`, both in Parquet format rather than CSV.

Why Parquet instead of CSV? Because Parquet is a columnar format, letting Spark read only the columns it actually needs (column pruning), in addition to much better compression than plain text CSV.

5.4 Task 4: `add_derived_columns`

Purpose: Enrich the cleaned data with derived columns that make later analysis easier, without having to recompute these values in every single analytical query.

The most important derived columns added:

- **Time-based columns:** Year, Month, DayOfWeek, IsWeekend (derived by parsing FlightDate into a real date), and Season (winter/spring/summer/fall, based on the month).
- **Delay-related columns:** IsDelayed (was the flight delayed by 15 minutes or more?), and DelayCategory (categorizing delays into: cancelled, unknown, on-time, minor delay, major delay, severe delay).
- **Route identifier (RouteID):** combining origin and destination airports into a single string like JFK-LAX.
- **Primary delay reason (PrimaryDelayReason):** among five columns representing different delay causes (carrier, weather, National Airspace System, security, late aircraft), the column with the highest value is chosen as the “primary reason” for that flight, using `greatest` and `coalesce` functions instead of any custom Python function (to avoid slowing down distributed processing).

After adding these columns, the result is saved as Parquet files **partitioned** by Year and Month, which significantly speeds up any later analytical query that filters data by a specific time period (partition pruning).

5.5 Task 5: `generate_aggregated_tables`

This is the most complex task in the pipeline, producing **five ready-made analytical tables** from the same enriched dataset, without re-reading it from disk five separate times (it’s read once and reused in memory via Spark’s execution plan). The five tables are:

1. **airline_performance** — per-airline performance: total flights, delayed and cancelled flight counts, average arrival/departure delay, and delay/cancellation rate percentages.
2. **route_performance** — per-route (flight path) performance: total flights, average delay, and average distance in miles for each route.
3. **airport_performance** — per-airport performance: departure statistics (as an origin airport) and arrival statistics (as a destination airport) are computed separately, then merged with a `FULL OUTER JOIN` on the airport code, so airports that only appear as an origin or only as a destination are still represented.
4. **monthly_trends** — monthly trends: total flights, delays, and cancellations per month/year/season.
5. **delay_reasons** — delay reasons: incident count and total delay minutes for each primary delay reason, broken down by year.

Important design note: all calculations here use standard `pyspark.sql.functions` (`groupBy`, `agg`, `when`, `coalesce`, `greatest`...) with no custom Python function (UDF) and no `.collect()` call, which allows Spark’s Catalyst engine to fully optimize the execution plan — critical when dealing with tens of millions of rows.

5.6 Task 6: `load_to_postgres`

Purpose: Load the five aggregated tables produced by the previous task (stored as Parquet files in HDFS) into a PostgreSQL database (`skylens-dw`), making them available to visualization tools.

How it works: This task creates a Spark session while loading the PostgreSQL JDBC driver file (postgresql-42.7.3.jar), then loops over each of the five tables, reads it from HDFS, and writes it to PostgreSQL using overwrite mode (so the table is recreated from scratch with the current data every time the pipeline runs).

This particular task was the source of one of the most significant problems we ran into during development, fully detailed in the “Main Problems and Their Solutions” section later in this report.

6 The Resulting Analytical Tables in PostgreSQL

After a successful full pipeline run, five tables were loaded into the `skylens_dw` database, with the following actual row counts (as confirmed both in the execution logs and directly via `psql`):

Table Name in PostgreSQL	Source in HDFS	Actual Row Count
<code>agg_airline_performance</code>	<code>airline_performance</code>	17
<code>agg_route_performance</code>	<code>route_performance</code>	7,559
<code>agg_airport_performance</code>	<code>airport_performance</code>	375
<code>agg_monthly_trends</code>	<code>monthly_trends</code>	36
<code>agg_delay_reasons</code>	<code>delay_reasons</code>	3

The existence and correctness of these tables was verified directly by connecting to the container and using the `psql` client:

```
docker exec -it skylens-dw psql -U skylens -d skylens_dw
```

then listing the tables (`\dt`) and confirming row counts in each table with simple `SELECT COUNT(*)` queries.

7 Connecting the Database to BI Tools

Once the analytical tables were ready inside `skylens_dw`, the next step was to visualize them. Two different tools were used for this purpose.

7.1 Connecting to Power BI

Since Power BI runs on the user's own machine (outside the Docker internal network), the connection goes through the port exposed to the host from the `skylens-dw` container:

Field	Value
Server	<code>localhost:5433</code>
Database	<code>skylens_dw</code>
Username	<code>skylens</code>
Password	<code>skylens_pass</code>

Important note about the port: inside `docker-compose.yml`, the service definition contains:

```
ports:
  - "5433:5432"
```

meaning PostgreSQL runs internally (inside the container and inside the `bigdata-net` network) on the standard port 5432, but it is port-mapped to 5433 on the host machine itself. Therefore, any connection from **outside** the Docker network (like Power BI running directly on Windows) must use `localhost:5433`, while any connection from **inside** the network (like Spark or Superset, both running as containers on the same network) uses `skylens-dw:5432` directly via the service's internal name.

Steps to connect in Power BI Desktop:

1. **Get Data** → search for **PostgreSQL database**.
2. Enter `localhost:5433` in the Server field and `skylens_dw` in the Database field.
3. Enter the credentials (`skylens` / `skylens_pass`) when prompted.
4. Select the five `agg_*` tables and load them.

Power BI requires the **Npgsql .NET Driver** to be installed on the machine in order to connect to PostgreSQL; if it isn't installed, an explicit error prompts for it.

7.2 Connecting to Apache Superset

Unlike Power BI, Superset was not originally part of the project environment, so it was added as a new service inside the same `docker-compose.yml` file, on the same internal network `bigdata-net`, so it could connect to `skylens-dw` directly via the service's internal name without needing any port mapping.

The service definition that was added:

```
superset:
  image: apache/superset:latest
  container_name: superset
  hostname: superset
  restart: unless-stopped
  depends_on: [ skylens-dw ]
```

```
environment:
  - SUPERSET_SECRET_KEY=change-me-please-superset-key
ports:
  - "8088:8088"
volumes:
  - superset_data:/app/superset_home
networks: [ bigdata-net ]
```

After running the container for the first time, it was set up through three one-time initialization commands:

```
docker exec -it superset superset db upgrade
docker exec -it superset superset fab create-admin \
  --username admin --firstname Admin --lastname User \
  --email admin@example.com --password admin
docker exec -it superset superset init
```

After that, the Superset UI became available in the browser at <http://localhost:8088>.

When adding the database connection from inside the Superset UI (Settings → Database Connections), the **internal** connection details were used (the service name and internal port, not `localhost:5433`):

Field	Value
Host	skylens-dw
Port	5432
Database name	skylens_dw
Username	skylens
Password	skylens_pass

This connection did not succeed on the first attempt, and ran into more than one technical obstacle that had to be diagnosed and resolved — fully detailed in the next section.

8 The Dashboard and the Five Charts

After the connection from Superset succeeded, a separate **Dataset** was created for each of the five tables, then one distinctive chart was built from each table, and all of them were collected into a single dashboard named “**Skylens Overview**”. The chart type was deliberately varied depending on the nature of the data in each table:

#	Source Table	Chart Type	What the Chart Shows
1	agg_airline_performance	Bar Chart	Total number of flights per airline (X-axis: airline, value: SUM(total_flights))
2	agg_monthly_trends	Line Chart	Trend of total flights across the twelve months of the year
3	agg_route_performance	Bar Chart	The top 10 busiest routes (sorted descending by SUM(total_flights))
4	agg_delay_reasons	Pie Chart	Percentage distribution of each primary delay reason
5	agg_airport_performance	Bar Chart	The top 10 busiest airports by total number of flights

8.1 Notes on Some of the Charts

- “**Top 10 busiest routes**” chart: since the source table contains more than 7,500 rows, it was necessary to use the **Series Limit** feature (set to 10) together with **Sort by** on SUM(total_flights) in descending order, to show only the top 10 routes instead of drawing thousands of unreadable columns.
- “**Delay reasons distribution**” chart: the source data for this chart contains only 3 rows in the table (agg_delay_reasons), and all of them share the same primary delay reason, “Carrier” (the airline’s own delay), so the pie chart shows one dominant color — this is a correct reflection of the actual data, not a chart configuration error.

9 Main Problems We Faced and How We Solved Them

This section documents the most important technical problems that came up while building the project, explaining the root cause of each one and the final solution applied, along with the lesson learned from each case.

9.1 Problem 1: `load_to_postgres` Task Failing With `ClassNotFoundException`

Symptoms: the `load_to_postgres` task (the sixth and final task in the DAG) kept failing repeatedly (after several automatic Airflow retries), and when the execution log for that task was inspected, the following message appeared inside the stack trace:

```
py4j.protocol.Py4JJavaError: An error occurred while calling o38.jdbc.  
: java.lang.ClassNotFoundException: org.postgresql.Driver
```

Diagnosis: the code specified the path to the PostgreSQL JDBC driver file as follows:

```
JDBC_JAR_PATH = "/opt/spark-apps/postgresql-42.7.3.jar"
```

and it was confirmed that this file did in fact exist inside both the `spark-master` and `spark-worker` containers (verified with a direct `ls -la` command inside each container). But the key question was: **where does this task actually run?**

Since Airflow runs with `LocalExecutor`, the Python code of every `PythonOperator` (including creating the Spark session itself as the Driver Process) executes inside the **airflow-scheduler** container itself, not inside `spark-master`. Checking the same path inside the `airflow-scheduler` container:

```
docker exec -it airflow-scheduler bash -c \  
  "ls -la /opt/spark-apps/postgresql-42.7.3.jar"  
# ls: cannot access '/opt/spark-apps/postgresql-42.7.3.jar':  
# No such file or directory
```

confirmed that the file **did not exist at all** inside the `airflow-scheduler` container, because `docker-compose.yml` only defined the same `spark/apps` volume mount for `spark-master` and `spark-worker`, and never for `airflow-scheduler`.

Permanent fix: the same volume mount was added to the `airflow-scheduler` service in `docker-compose.yml`:

```
airflow-scheduler:  
  volumes:  
    - ./airflow/dags:/opt/airflow/dags  
    - ./airflow/logs:/opt/airflow/logs  
    - ./airflow/plugins:/opt/airflow/plugins  
    - ./spark/apps:/opt/spark-apps # <-- line added
```

then the container was recreated with the new configuration:

```
docker-compose up -d --force-recreate airflow-scheduler
```

After this change, the JAR file became permanently available inside the scheduler container, even if the container is recreated in the future, and the full task success was verified by running it manually:

```
docker exec -it airflow-scheduler bash -c \  
  "airflow tasks test skylens_batch_pipeline load_to_postgres 2026-01-01"
```

which produced the final success message: `load_to_postgres()` complete. All 5 tables loaded into `skylens_dw`.

Lesson learned: when using `LocalExecutor` in Airflow, you must always think about *which container specifically* will execute the logic of each `PythonOperator`, rather than assuming that any path available in the Spark containers will automatically be available in the Airflow container too, even if they share the same network.

9.2 Problem 2: Could not load database driver for: postgresql Error in Superset

Symptoms: when trying to add a new PostgreSQL database connection from the Superset UI, an immediate error appeared:

ERROR: Could not load database driver for: postgresql

Diagnosis (step by step):

1. The first attempt was to install the required library (`psycopg2-binary`) via a normal `pip install` inside the container, and the command showed Requirement already satisfied — meaning the library was “installed” from `pip`’s point of view.
2. Despite that, when trying to import the library directly through Python:

```
docker exec -it superset python3 -c "import psycopg2; print('OK')"  
# ModuleNotFoundError: No module named 'psycopg2'
```

an error appeared saying the library didn’t exist at all! This apparent contradiction was worth investigating.

3. Upon inspection, it turned out the reason for this contradiction was that **two completely different Python environments** were being used inside the same container:
 - The actual `python3` points to `/app/.venv/bin/python3` (an isolated Virtual Environment specific to the official Superset image).
 - While the regular `pip` points to `/usr/local/bin/pip`, which belongs to a **completely different** Python installation outside the virtual environment.
 - As a result, installing via the regular `pip install` placed the library in `~/.local/lib/python3.10/site-packages`, belonging to the wrong environment, while the actual Superset process runs from the `venv` that didn’t contain this library at all — and it turned out the `venv` itself didn’t even have a `pip` tool installed in the first place.

Solution: first install `pip` inside the correct `venv` environment via Python’s built-in `ensurepip` module, then use that specific `pip` (via its full path) to install the library in the right place:

```
docker exec -u root -it superset /app/.venv/bin/python3 -m ensurepip  
docker exec -u root -it superset /app/.venv/bin/python3 -m pip install \  
psycopg2-binary
```

and the fix was verified directly:

```
docker exec -it superset python3 -c "import psycopg2; print('OK')"  
# OK
```

After that, the container was restarted (`docker restart superset`) so that Superset’s web server process would pick up the newly installed library, since it had already been running before the installation.

Lesson learned: a pip message saying Requirement already satisfied doesn't necessarily mean the library is actually available to the process that will use it — especially in official Docker images that rely on isolated Python virtual environments for security or organizational reasons. The only real verification is to try importing the library directly through the same Python interpreter (python3) that the actual process uses.

9.3 Problem 3: SSL Error When Connecting Superset to PostgreSQL

Symptoms: after fixing the library issue, a completely different new error appeared when trying to connect:

```
(psycopg2.OperationalError) connection to server at "skylens-dw" (172.18.0.x),  
port 5432 failed: server does not support SSL, but SSL was required
```

Diagnosis: this error shows that the connection actually reached the server successfully (the internal IP address appears clearly in the message), but Superset was requiring SSL encryption to be used, while the standard postgres:13 container used for the skylens-dw service doesn't run in SSL mode at all (which is unnecessary anyway for a test environment running entirely inside a closed local Docker network).

Solution: add an extra parameter (sslmode=disable) when defining the connection in Superset, either in the “Additional Parameters” field or as part of the connection string (SQLAlchemy URI):

```
postgresql://skylens:skylens_pass@skylens-dw:5432/skylens_dw?sslmode=disable
```

After this change, the connection succeeded for the first time.

Lesson learned: some versions of Superset (and its accompanying SQLAlchemy libraries) assume by default that external databases support SSL, a reasonable assumption for production cloud environments but not suited to a simple local development environment; connection errors that explicitly mention SSL should always prompt checking the SSL settings.

9.4 Problem 4: Errors Building the First Charts (Mixing Up X-axis, Metrics, and Dimensions)

Symptoms: when building the first chart (for the airline performance table), an illogical chart appeared with randomly colored columns and strange values on the horizontal axis (numbers like 88791 and 233362 instead of airline names).

Diagnosis: the initial chart configuration mixed up three core concepts in Superset:

- **X-axis:** mistakenly contained a numeric column (total_flights) instead of the categorical column (Reporting_Airline), causing the chart to treat every unique numeric value as a separate “category”.
- **Dimensions:** contained Reporting_Airline, which caused the data to be split and colored separately for every single airline unnecessarily.
- **Metrics:** used COUNT(*) instead of SUM(total_flights), and since every row in this table already represents a single airline, the COUNT(*) result for any group would always be 1 — a useless number.

Solution: redistribute the fields correctly:

- **X-axis** ← Reporting_Airline (the categorical column).
- **Metrics** ← SUM(total_flights) (the actual measure to display).
- **Dimensions** ← left completely empty (since the only categorization needed is already present in X-axis).

After this correction, the chart appeared correctly: one clear column per airline, with a height representing its total number of flights.

Lesson learned: the difference between X-axis and Dimensions in modern BI tools like Superset isn't always obvious to a new user; as a general rule, the main categorical column you want to break the data down by should

appear in **one place only** — either X-axis (for a simple single-dimension chart) or Dimensions (for additional breakdown inside a more complex chart) — putting it in both causes duplication and confusion in the result.

9.5 Problem 5: Duplicate column/metric labels Error

Symptoms: while trying to add color variety to the columns in one of the charts (making each column a different color), the same column already used in X-axis (AirportCode) was added again in the Dimensions field, producing the error:

Duplicate column/metric labels: "AirportCode".

Please make sure all columns and metrics have a unique label.

Diagnosis: Superset refuses to use the same column with the same label in more than one field within the same query, because that would produce two identical labels in the final query result, which conflicts with the tool's design.

Solution: the simplest and visually cleanest option was chosen — **not duplicating the column at all**, leaving the Dimensions field empty, and relying instead on the default color system in the “Customize” tab (Color Scheme) to give the chart a clean, professional, uniform look, instead of trying to force a per-column random coloring, which technically requires an **additional** categorical column (different from the X-axis column) that isn't logically available in this particular table.

Lesson learned: not every dataset is suited to coloring each column a different color; if there's no meaningful additional categorical column, using a single uniform color or a single color gradient from the Color Scheme is the more visually appropriate and technically correct choice.

10 Summary of General Lessons Learned

Looking at all the problems above together, several general lessons can be drawn that would benefit any similar Big Data project:

1. **Understand where the code actually runs:** in distributed systems (multiple containers, different Executors), an error message like “file not found” or “library not found” first requires knowing *exactly which process* tried to access that file or library, before searching for a solution.
2. **Don’t trust the “success” of an install command alone:** a successful `pip install` doesn’t mean the library is available to the actual running process; real verification always comes from trying to actually use the library or file from the same context (same user, same Python environment, same container) it will actually run in.
3. **Read the full error message before assuming the cause:** a lot of time can be wasted guessing at possible causes of a problem, while the full error message (not just its first line) often states the root cause explicitly (like `ClassNotFoundException` or `server does not support SSL`).
4. **Temporary fixes are not final solutions:** manually editing a file inside a container (`docker cp`) may solve the problem momentarily, but it gets lost when the container is recreated; the correct permanent fix is usually in the `docker-compose.yml` file itself (like adding a permanent volume mount).
5. **Understand BI tools before using them randomly:** every field in a tool like Superset (X-axis, Metrics, Dimensions) has a specific purpose; using a field incorrectly doesn’t necessarily produce an explicit software error, but it does produce a visually illogical chart that’s hard to interpret without understanding the purpose of each field.

11 Conclusion

The Skylens project presents a complete, realistic example of an end-to-end Big Data pipeline, starting from storing large raw data in HDFS, going through distributed cleaning and processing stages via Apache Spark, fully managed through automatic scheduling with Apache Airflow, and ending with a relational data warehouse (PostgreSQL) accessible from more than one visualization tool at the same time (Power BI and Apache Superset).

The real value of this project lies not just in getting the pipeline to run successfully on the first try, but in the realistic problems that came up during development — from networking and volume issues between containers, to Python environment conflicts inside ready-made Docker images, to SSL settings, all the way to correctly understanding visualization tools — which is exactly the kind of problem any data engineer faces in a real work environment, not just a theoretical exercise.